

Programming Assignment 3 – Computer Networks

Teachers: Fatima Zahra Errounda, Stephan Schiffel

September 4, 2026

Question	Points	Score
1	80	
2	10	
3	10	
4	10	
Total:	110	

Logistics

- This is a group assignment, you should work in groups of 2 to 3 students.
 - **Source code for programming assignments must be submitted in a single zip file, with accompanying build instructions that work. Anything else will receive an automatic 0.**
 - All code must be written in C or C++
 - The code must be accompanied by a Makefile, such that running make on the command line is enough to compile it.
 - A README file is included mentioning:
 - * which OS/machine you developed on (Windows, MacOS on Intel, MacOS on ARM, Linux, ...)
 - * how to compile and run the submitted code
 - * which assumptions (if any) you made when developing
 - * which parts of your code don't work (if any)
 - Code is clearly structured and well commented. Each function/class has a header describing its purpose.
- You will need to program and run your program on your own computer. Some part of this assignment will require that you run your code as root. Thus it is important that you understand what your code does and/or take precautions (such as run it in a virtual machine).

Speak easy to the port, and perhaps it will let you in.

In this assignment you will be introduced to the delights of packet crafting, bit twiddling and UDP subterfuge.

Somewhere on the TSAM server (130.208.246.98), a server is listening to some UDP ports in the range 4000-4100. Find the ports, send them the right packets, and use the secret knock to gain access!

1. (80 points) **Puzzle Solving**

Using your port scanner from Assignment 1 (or nmap), you should be able to discover 4 open UDP ports on the TSAM server in the 4000-4100 range. The ports are puzzle ports, safeguarding information about two additional ports (the secret ports) which are not showing up on your scan. Your task is to write a program to solve the puzzles, in order to reveal the two secret ports, a secret phrase and finally use this information to knock on the final port. Each port will send you instructions on how to reveal its secret if you send it a UDP message that is at least 6 characters long.

The program must be run with the command:

```
./puzzlesolver <IP address> <port1> <port2> <port3> <port4>
```

Notes:

- The puzzle ports and their order will change over time, so do not hard code the ports, but rather supply them as command line arguments to your program.
- **The program must not make any assumptions about the order of the ports given** (e.g., you can not assume the first argument to always be a the same puzzle port). Instead, it must detect which port is which from the responses it gets to some default message.
- Any information that you get as a response from the puzzle ports can change over time and must not be hard-coded! Rather, your program must extract the information from the responses and store it in some variables to use later.
- The only thing hard-coded in your program should be the user names of your group members.

Points are distributed for solving:

(a) (20 points) S.E.C.R.E.T. port

- (b) (20 points) Evil port
 - (c) (20 points) "Guardian of the secret spell" port
 - (d) (20 points) D.R.A.G.O.N. + port-knocking
2. (10 points) **Code Quality** Points will be awarded for code quality, structure, commenting and quality of the Readme file.
 3. (10 points) **Explaining Your Solution** You will be asked in class to explain your solution and some details of the implementation.
 4. (10 points) **Bonus** After completing the port-knocking, you are sent a secret message, follow the instructions in the secret message for some bonus points.

Tips and Hints

- Most of the puzzles can be solved by using a normal UDP socket like you used in Assignment 1, only one needs a raw socket.
- Wireshark is your friend! Always check carefully if the packet that is sent is what you wanted to send. Always check the packets you receive in wireshark and to see whether your program missed or misinterpreted something.
- Every bit is important. Follow the instructions exactly. Don't add extra bytes in your messages and set all values exactly as required.
- Check the byte-order!
- Ask for help if you are stuck, don't be stuck for too long.
- Check out the hints page for the assignment on Canvas.